NTILE Functions

The **NTILE()** function is a window function in that divides the result set into a specified number of roughly equal groups (or buckets) and assigns a number to each row indicating its group. It's commonly used for **ranking** data and for **distributing data evenly** across groups.

Syntax:

NTILE(number_of_groups) OVER (PARTITION BY column ORDER BY column)

- **number_of_groups**: The number of groups you want to divide the result set into.
- **PARTITION BY**: Optional. Divides the result set into partitions.
- **ORDER BY**: Specifies the order in which rows are assigned to each group.

5 Examples Using NTILE()

---

1. Divide Customers into 4 Equal Groups Based on Their Purchase Amount

Scenario: You want to divide customers into 4 groups based on their total purchase amounts, with the group number assigned to each customer.

Query:

```
SELECT
   customer_id,
   SUM(quantity * cost_to_customer_per_qty) AS total_purchase,
   NTILE(4) OVER (ORDER BY SUM(quantity * cost_to_customer_per_qty) DESC) AS purchase_group
FROM
   customer_purchases
GROUP BY
   customer_id;
```

Explanation:

- The NTILE(4) divides customers into 4 groups based on their total purchase amount.

- The highest purchasers go into the highest group, and the lowest purchasers go into the lowest group.

---

2. Rank Vendors into 5 Groups Based on Their Revenue

Scenario: You want to rank vendors into 5 revenue groups, with the highest revenue vendors in group 1 and the lowest in group 5.

Query:

```
SELECT
    vendor_id,
    SUM(quantity * cost_to_customer_per_qty) AS total_revenue,
    NTILE(5) OVER (ORDER BY SUM(quantity * cost_to_customer_per_qty) DESC) AS revenue_group
FROM
    customer_purchases
GROUP BY
    vendor_id;
```

Explanation:

- The NTILE(5) divides vendors into 5 groups based on their revenue.
- Vendors with the highest revenue are ranked in group 1, and those with the lowest in group 5.

---

3. Divide Product Sales into 3 Equal Buckets for Each Product Category

Scenario: You want to group product sales into 3 buckets for each product category, sorted by the quantity sold.

Query:

```
SELECT
    product_category_id,
    product_id,
    SUM(quantity) AS total_quantity_sold,
```

```
    NTILE(3) OVER (PARTITION BY product_category_id ORDER BY SUM(quantity) DESC) AS
sales_bucket
FROM
    customer_purchases
JOIN
    product ON customer_purchases.product_id = product.product_id
GROUP BY
    product_category_id, product_id;
```

Explanation:

- NTILE(3) divides the products in each category into 3 groups based on the total quantity sold.
- This allows you to see which products perform best, moderately, or worst within each category.

---

4. Segment Customers into 2 Groups Based on Their Purchase Frequency

Scenario: Divide customers into 2 groups based on how frequently they make purchases.

Query:

```
SELECT
    customer_id,
    COUNT(*) AS purchase_frequency,
    NTILE(2) OVER (ORDER BY COUNT(*) DESC) AS frequency_group
FROM
    customer_purchases
GROUP BY
    customer_id;
```

Explanation:

- The NTILE(2) function divides the customers into 2 groups based on the number of purchases.
- Customers with the highest purchase frequency are in group 1, and those with the lowest are in group 2.

---

5. Classify Market Dates into Quartiles Based on Total Revenue

Scenario: You want to classify market_dates into quartiles (4 groups) based on total revenue generated on each date.

Query:

```
SELECT
    market_date,
    SUM(quantity * cost_to_customer_per_qty) AS total_revenue,
    NTILE(4) OVER (ORDER BY SUM(quantity * cost_to_customer_per_qty) DESC) AS revenue_quartile
FROM
    customer_purchases
GROUP BY
    market_date;
```

Explanation:

- The NTILE(4) divides market_dates into 4 quartiles based on the total revenue generated.
- This helps to understand which dates are top-performing in terms of sales and which are not.

Key Takeaways:

- **NTILE()** is useful for dividing a dataset into a specified number of equal groups.
- It can be used for creating **rank buckets**, **quartiles**, **percentiles**, etc.
- It requires an ORDER BY clause to determine the order in which rows are assigned to groups.